Sprint 2

Peyton Durand, Taylor Tran, Riley Marek

Introduction

Adapt-AI Online Community & Professional Learning

- Collaborative environment for educators to share, discover, and reuse educational resources

Team Roles

- Peyton Durand: Team Lead
- Taylor Tran
- Riley Marek

Sprint 2 Goals

- Implement core functionality of the community feature
- Essential to building an engaging and interactive platform

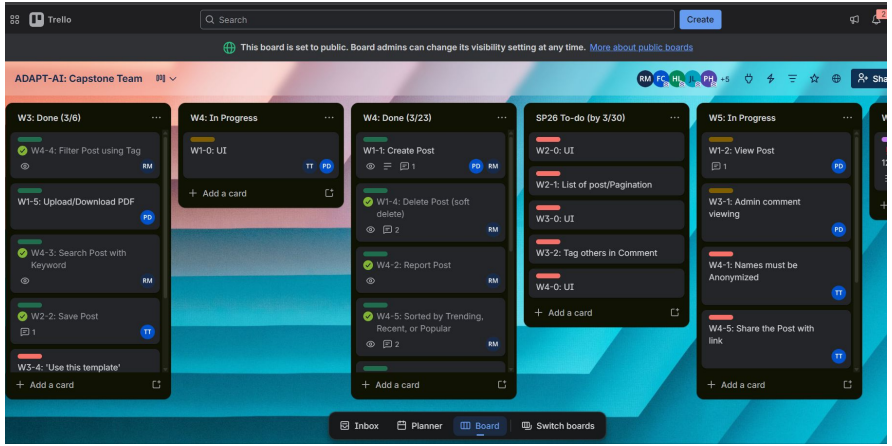
Sprint Objectives

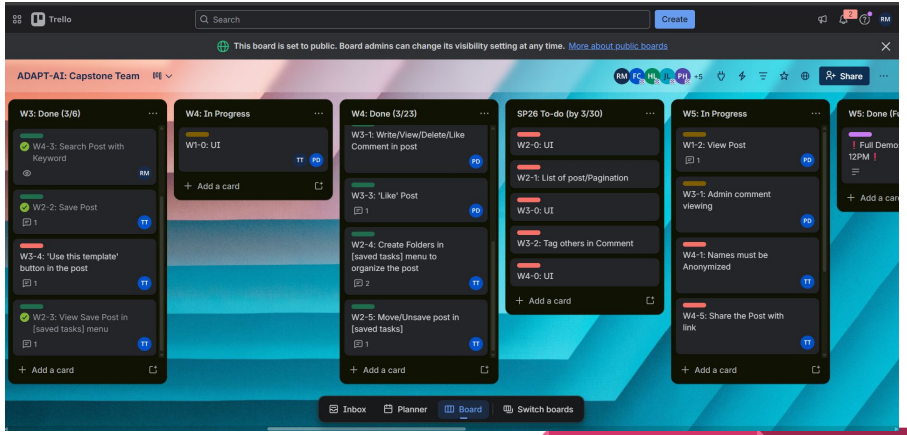
- Implement full community page functionality.
- Maintain strong communication within the Capstone team.
- Produce strong results and impress the client.

Feature Implementation & Team Member Contributions

1. Updated Post UI and Post Editor - Riley, Peyton, Taylor
2. Liking Posts Feature - Peyton
3. Commenting Posts Feature -Peyton
4. Liking Comments Feature -Peyton
5. Embedded PDF feature (paginated) - Peyton
6. Paginated Posts feature -Payton
7. Saving/Unsaving Posts Feature - Taylor
8. Creating Folders Feature - Taylor
9. Report Feature - Riley
10. Edit Post & Soft Delete Post feature - Riley
11. Searching Post feature - Riley
12. Filtering (New, Popular, My Posts) feature - Riley

Kanban Overview & Team Contributions





Demo Video

https://youtu.be/nuFzyt4kLpc

Testing & Validation

1. Dev vs production .env strategy. Testing on local postgres database, dev (local host) frontend, dev Azure storage.
2. Difficult to unit test or integration test, as main of the core application components are fully reliant on other features to work properly. Most of our testing is system wide, where we test that the application works as intended from a end user standpoint, doesn’t crash, and that individual features work properly within the system.

Documentation of System architecture, database, user interface

a) System architecture
   i) General data flow
      1) (User information, post information, comment information, etc) flows from the frontend to the backend, where it gets stored in the database
      2) Upon event triggers (such as delete or report button) the backend updates the database (soft delete) and the event flows up to the frontend, where the post is no longer visible
   ii) Three main components:
      1) Frontend
         (a) TypeScript, React 18, Vite, TailwindCSS, Radix UI
         (b) Used react for its dynamic and modern interfaces
         (c) Vite optimizes the frontend build and catches errors
         (d) Typescript for added type security and code maintainability
      2) Backend
         (a) Python 3.11+, FastAPI, Uvicorn, Pydantic
         (b) FASTAPI modern framework, tight python debugging integration
         (c) Uvicorn for speed and hot reload, watchfiles for faster development
      3) Database/infrastructure
         (a) PostgreSQL 16, SQLAlchemy, Azure Blob Storage, Azure AI Search Docker
         (b) PostgreSQL has optimized query operations and scales well
         (c) Azure enterprise grade cloud storage, and AI search is highly optimized for searching through pdf knowledge bases, while Blob stores massive amounts of raw files
b) Database design
   i) Show DBeaver tables and diagrams
   ii) Indexing and optimization strategies: deleted_at filters rows out, Azure AI Search (optimized pdf searching)
   iii) ILIKE searches for partial matches, (less efficient, but allows the users to search for posts without knowing the exact wording of a title or author name)
   iv) Post tags for subject and grade level are structured database filters (JSONB array of existing filters), so that the database avoids unnecessary work and has to search through less items if a tag already exists
c) User interface design
   i) initial design (simple editor, lacking features, simple card layout with title and text), discuss client feedback and website colleague ai
   ii) Current changes to post layout, title card layout and full Page editor

Responsible use of AI

Tools used during the sprint:

- Claude Sonnet, OpenAI Codex

How we used AI:

- Brainstorming UI design
- Assisting with code generation and debugging

Improvements:

- Faster development workflow
- Better structured code

Limitations:

- Occasional incorrect outputs
- Manual verification and testing

Sprint achievements and challenges

Achievements:

- Successfully implemented key interaction and engagement features

Challenges and Solutions:

- Integrating new features into the already existing system. We solved this by separating concerns between the frontend and backend, and using consistent API structures to coordinate
- Managing feature complexity as more interaction were added. By prioritizing incremental development and testing each feature first, we solved this problem.

Progress from Sprint 1

- Sprint 1 focused more on foundational setup and basic functionality
- Sprint 2 is a more complete and interactive platform where users can create content and also interact with others

Next Steps and Sprint Retrospective

- Prepare final sprint demo for client.

- Receive instructions for next sprint (still unknown).

Conclusion

- Thank you to the CAIRE team and Professor Kumar.